

강화학습(PPO) Snake 실습 보고서

YSAL 8/9기 Lecture Session 8 (Reinforcement Learning) · 박동윤 · 2026.09.27

구분	내용
선택한 브랜치	main (README 권장: 학습 속도가 가장 빠름)
과제 구성	1) main 브랜치 기본값으로 from scratch 학습 2) 세 브랜치(main/14dim-multicore-branch/update-state-with-relative-direction)의 get_state() 차이 분석 3) main 브랜치 보상 체계를 임의로 바꿔 재학습
1) 기본 학습 결과	last500 평균 사과 수 39.1, 새 평가 30판 평균 46.2(최저 27~최고 71)
2) 상태 비교 결론	상태에 상대적 정보(진행방향 기준 사과 방향)와 공간·거리 센서를 더한 update-state-with-relative-direction(13차원)이 같은 7000 에피소드에서 가장 높았음(108.9). 다만 두 비교 브랜치는 보상 체계도 함께 바뀌어 있어 상태만의 순수한 효과는 아님
3) 보상 변경 결과	목표를 '사과를 피해 오래 생존하기'로 바꾸고(사과=페널티, 거리보상 반전) 재학습한 결과, 30판 전부 사과를 0개 먹음 (의도한 대로 회피 행동 학습)

1. 실습 개요

선택한 브랜치: main / 14dim-multicore-branch / update-state-with-relative-direction 중 README가 권장한 main 브랜치를 선택했습니다. 아래 2번(상태 비교)에서는 나머지 두 브랜치도 각 브랜치 자신의 main.py로 직접 학습해 비교했습니다.

환경: Snake 게임, 행동 3개(직진·좌회전·우회전). main 브랜치 기본 보상은 사과 +10, 벽/몸통 충돌 또는 굶주림 종료 -10, 매 스텝 -0.01, 사과와 가까워지면 +0.1 / 멀어지면 -0.1 입니다.

알고리즘/모델: PPO Actor-Critic(은닉 64, Tanh), 2000 스텝마다 모은 데이터로 정책을 갱신합니다 (main 브랜치 기본값 기준. 다른 두 브랜치는 8개 환경을 동시에 돌리는 멀티프로세스 구조이며 update_timestep 도 8배 규모로 데이터를 모읍니다).

실행 환경: macOS, Python 3.13, PyTorch 2.13(CPU). 학습은 SDL_VIDEODRIVER=dummy 로 화면 없이 진행했고, 비교 지표는 학습 마지막 500 에피소드 평균 사과 수(last500)와 학습이 끝난 최종 모델을 새 난수 시드로 30판 평가한 평균 사과 수입니다.

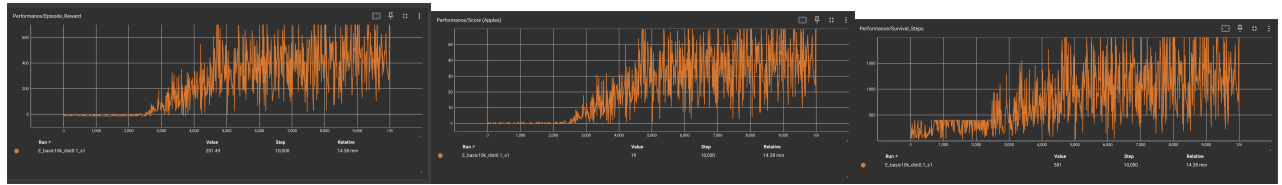
2. main 브랜치 기본값 학습 (from scratch)

main.py 를 수정 없이(코드 인자화는 했지만 기본값은 원본과 동일) 그대로 실행했습니다.

항목	값
state (상태 차원)	basic, 11개 (위험 3 · 이동방향 4 · 사과방향 4, 모두 절대 좌표 기준)
dist_reward / lr / gamma	0.1 / 0.0003 / 0.99 (기본값, 변경 없음)
epochs / eps_clip / update_timestep / entropy_coef	4 / 0.2 / 2000 / 0.01 (기본값, 변경 없음)
episodes / seed	10000 (기본값) / 0
모델 파일	saved_models/E_basic10k_dist0.1_s1/ppo_snake_final.pth

결과: last500(마지막 500 에피소드) 평균 사과 수 39.1개, 총 학습 시간 약 14.4분(10000 에피소드). 학습이 끝난 모델을 이전에 쓰지 않은 새 난수 시드로 30판 평가한 평균은 46.2개(최저 27 ~ 최고 71)였습니다.

[그림 1] TensorBoard 학습 로그 (E_basic10k_dist0.1_s1, 10000 에피소드)



Episode_Reward

Score (Apples)

Survival_Steps

2500 에피소드 이후 점수와 생존 스텝이 함께 오르며 학습이 진행되는 모습입니다. play.py 실행 영상은 별도 파일(submission/)로 첨부했습니다.

3. 세 브랜치의 get_state() 차이 분석

세 브랜치의 snake_code.py 를 비교하면 상태(에이전트가 행동을 고를 때 보는 정보)의 차원과 내용이 다릅니다. update-state-with-relative-direction 은 보상 체계도 14dim-multicore-branch 와 함께 바뀌어 있어, 상태 차이만의 순수한 효과는 아래 결과에 섞여 있습니다.

3.1 코드 비교

브랜치	차원	구성	보상 체계
main	11	위험 3(절대: 직진·우·좌) + 이동방향 4(절대) + 사과방향 4(절대)	기본값 (사과 +10, 사망 -10, 거리 ± 0.1 , 스텝 -0.01)
14dim-multicore-branch	14	main과 동일한 위험/방향/사과 정보 + 공간 센서 3개(직진·좌·우로 갔을 때 BFS로 전 도달 가능 칸 수)	사과 +1(거리보상 누적), 사망 -1.5, 거리 ± 0.01 . 뱀 길이 20 이상부터 다음 행동이 같히는 곳이면 미리 종료(-1.5)
update-state-with-relative-direction	13	위험 3(절대) + 사과 방향 4(진행방향 기준 상대: 직진·후방·좌·우) + 공간 센서 3 + 장애물까지 거리 3. 절대 이동방향(4개)은 제거	14dim-multicore-branch와 동일

공통적으로 두 비교 브랜치는 8개 환경을 병렬로 돌리는 multi_env_processor.py 구조를 쓰고, 뱀 길이 20 이상부터는 다음 행동이 이어지는 공간을 미리 BFS로 계산해 같히는 곳이면 그 행동을 하기 전에 에피소드를 종료하는 방어 로직이 추가되어 있습니다(main에는 없음).

3.2 실행 결과 (동일 조건: 7000 에피소드, seed 0, 처음부터 학습, 새 시드로 30판 평가)

각 브랜치를 브랜치 자신의 main.py로 직접 처음부터 학습시켜(episode 수만 7000으로 통일) 공정하게 비교했습니다.

브랜치 (상태 차원)	last500 / 평가 30판 평균	최저~최고	7000ep 학습 시간
main (11)	48.1 / 46.9	9 ~ 77	약 10.8분
14dim-multicore-branch (14)	-* / 58.8	24 ~ 86	약 22.0분
update-state-with-relative-direction (13)	-* / 108.9	50 ~ 152	약 38.9분

* 두 브랜치는 8개 환경이 동시에 끝나는 시점이 달라 정확히 마지막 500 '에피소드'만 골라내기 어려워 last500 은 생략하고 30판 평가로만 비교했습니다.

참고로 각 브랜치에는 과제 저장소에 이미 올라와 있던 예시 모델이 있어(14dim: 25000 에피소드/score45, update-state: 20000 에피소드/score125), 같은 30판 평가를 해봤습니다: 14dim 제공 모델 평균 53.3(12~81), update-state 제공 모델 평균 128.0(87~167)로, 7000 에피소드 결과와 같은 순서(update-state > 14dim)를 보였습니다.

3.3 분석

- update-state-with-relative-direction 이 가장 높았습니다. 사과 방향을 절대 좌표가 아니라 진행 방향 기준 상대 방향으로 주는 방식이, '직진하면 가까워지는지'를 신경망이 훨씬 적은 변환으로 바로 알 수 있게 해준 것으로 보입니다.

main/14dim은 이동방향(4)과 사과의 절대방향(4)을 각각 주고 신경망이 그 조합에서 '상대적으로 어느 쪽인지'를 스스로 학습해야 합니다.

- 14dim-multicore-branch는 공간 센서만 추가되고 사과 방향은 여전히 절대 좌표라서, main보다는 나아지지만(46.9→58.8) update-state만큼 크지는 않았습니다.
- 다만 이 비교에는 상태 외의 변수(보상 체계, 병렬 환경 수, 뱀 길이 20↑ 초기 종료 로직)가 함께 섞여 있어, 위 순위가 상태 표현만의 효과라고 확정할 수는 없습니다. 예를 들어 사망 패널티가 -10→-1.5로 작아진 것만으로도 죽음에 대한 부담이 줄어 다른 학습 양상을 보일 수 있습니다.
- 이전 실습(Lecture 1)에서 직접 시도했던 '몸통 정보 추가(도달 가능 공간·장애물 거리·꼬리 위치)' 실험에서도 비슷한 결론을 얻었습니다: 상태에 공간 정보를 더하면 점수가 오르지만, 그것만으로 갇혀 죽는 비율이나 비효율적인 경로가 사라지지는 않았습니다.

4. 보상 체계 변경 (main 브랜치, 목표를 '사과 회피'로)

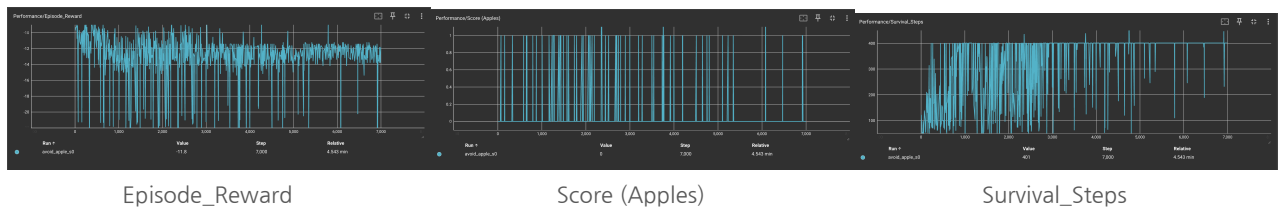
과제 예시(사과를 먹으면 페널티)를 그대로 적용했습니다. main.py/snake_code.py에 `reward_mode` 옵션을 추가해, 기존 동작(default)은 그대로 두고 `avoid_apple` 를 선택하면 아래처럼 목표가 뒤집히도록 했습니다.

요소	기존(default)	변경(avoid_apple)
사과 획득	+10	-10 (페널티)
거리 보상 (가까워짐/멀어짐)	+0.1 / -0.1	-0.1 / +0.1 (부호 반전)
스텝 패널티 / 사망 패널티	-0.01 / -10	변경 없음
나머지 하이퍼파라미터	lr 0.0003, gamma 0.99, epochs 4, eps_clip 0.2, update_timestep 2000, entropy 0.01, episodes 7000, seed 0 (모두 기본값과 동일, 처음부터 학습)	

결과: 학습 마지막 500 에피소드 평균 사과 수는 0.024개(사실상 0)였습니다. 새 난수 시드로 30판을 평가한 결과 30판 모두 사과를 0개 먹었고, 평균 401스텝(뱀 길이 3 기준 굶주림 종료 시점)까지 생존하다 게임이 끝났습니다. 즉 뱀이 사과를 능동적으로 피하도록 학습되었습니다.

모델 파일: `saved_models/avoid_apple_s0/ppo_snake_final.pth`

[그림 2] TensorBoard 학습 로그 (avoid_apple_s0, 7000 에피소드)



사과 획득(Score)이 학습 초반부터 0에 붙고, 생존 스텝은 뱀 길이 3 기준 굶주림 종료 시점인 약 400스텝에서 유지됩니다. 즉 사과를 피하면서 굶어 죽기 직전까지 버티도록 학습되었습니다. play.py 실행 영상은 별도 파일(submission/)로 첨부했습니다.

5. 제출 파일 구성

아래 내용을 하나의 zip으로 묶어 제출합니다.

- 이 보고서 (PDF)
- 1) 기본 학습 모델: `saved_models/E_basic10k_dist0.1_s1/ppo_snake_final.pth`
- 3) 보상 변경 모델: `saved_models/avoid_apple_s0/ppo_snake_final.pth`

- 두 모델 각각의 play.py 실행 영상 (TensorBoard 학습 로그는 이 보고서 [그림 1][그림 2]에 포함)

파일명/메일 제목: [Lecture8] 8/9기_곽동윤, 제출: yonseiysal@gmail.com, 기한: 9/28(월) 23:59

코드: github.com/DongYun22/9th_RL_Practice (snake_code.py의 reward_mode/state_mode 옵션, run_experiments.py)